SOFTWARE SUPPLY CHAIN SECURITY

Supply Chain Security in the Age of the AI Hacker

Defending the Enterprise at Machine Speed

Stephen Husak
Staff Engineer, Capital One

THE ASYMMETRIC SHIFT

The economics just flipped

THE OLD CLOCK

- **Researcher-weeks** to find a flaw. Longer to weaponize.
- Remediation windows scaled to severity. Periodic dependency reviews.
- Unwritten assumption: when a vuln goes public, you have time.

THE NEW CLOCK

- **Model-minutes.** Discovery was the bottleneck. It's gone.
- Claude Mythos, GPT-6 Astra reason over code, not just patterns.
- The capability is symmetric, but the attacker only needs one path that works.

Finding the flaw was always the expensive step. **That step just got cheap.**

WHAT ACTUALLY CHANGED

It's not speed. It's reasoning.

- **Legacy static analysis** was always cheap, and always shallow. It flags what looks dangerous.
- **Reasoning models** follow a value between functions and work out what an outside caller controls.
- They catch what manual review misses for years, because it takes understanding what code does.
- **Chaining:** stitching several harmless-looking bugs into one working attack.

Parser (3 levels deep)
Length check = exact bytes. An unchecked ×4 multiply overflows, wrapping the count to a small number.

Library above it
Pass-through. Forwards raw request data without validating it.

Caller above that
Allocates from the wrapped count, copies the real length.

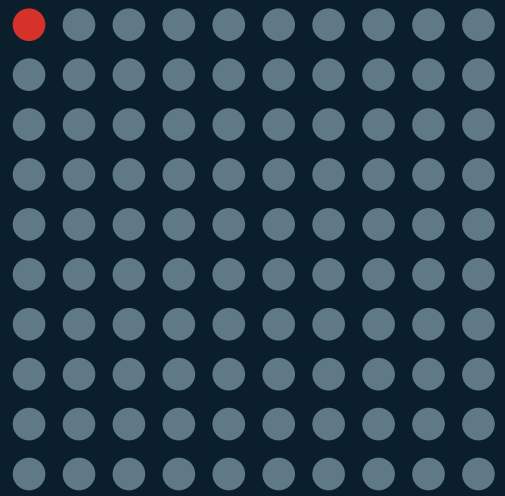
= Heap overflow. “Each looks defensible on its own.”

THE SORTING COST

Capable and noisy at once

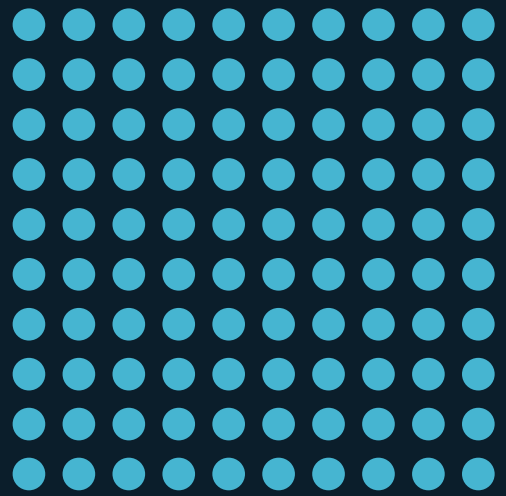
The whole long tail is now in scope, and both sides work from the same 100 findings.

ATTACKER



Needs 1 in 100 to hold up. Discarding the other 99 is effortless.

DEFENDER



Owns all 100. Every wrong one consumes an engineer.

Identical output. Cheap for one side, expensive for the other. The gap is the sorting cost rather than the tooling.

WHAT IT BREAKS

What the asymmetry breaks



The fix-exists-to-fix-deployed gap is the whole attack window, and most programs never measure it.

Publication as a milestone

Programs track when upstream releases a fix. The number that matters is when it runs in production. A published patch is a map for attackers.

Maintainer triage

Volunteers face many confident, professional-looking AI reports. Many are real, others point at code that never runs, and reading each one takes longer than generating it.

THE PROBLEM, LOCALIZED

The Transitive Graveyard

40 libraries an engineer can name. **400** the build system reports.



The ones you chose, and the ones you did not. Nobody approved them, nobody reviews them, and they run with first-party privileges.

WHAT “UNMAINTAINED” MEANS FOR YOUR RISK

The fix chain that never starts

WELL-RUN PROJECT



ABANDONED PACKAGE

The chain never starts. No release to upgrade to. The flaw is confirmed, public, and permanent. We call it a **forever-day.**

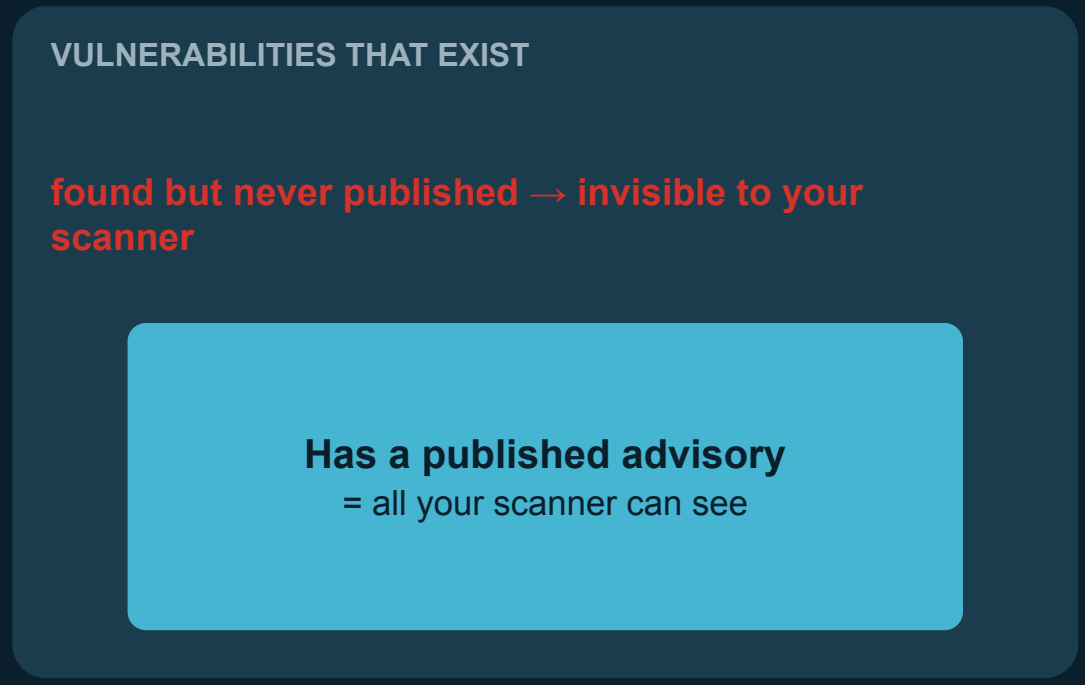
The economics favor the attacker twice:

dormant code is easier to find bugs in, and those bugs remain useful indefinitely. A flaw in a busy framework gets patched in days. The same flaw in a dead utility gets nothing.

THE DEPENDENCY NOBODY WILL FIX

The green dashboard is lying to you

- **Software composition analysis** matches installed versions against published advisory feeds.
- An attacker running discovery has no reason to publish. The finding is worth more unlisted.
- **No advisory to match,** so the scan comes back empty. Inventory complete, dashboards green.



A clean report means nobody has ***published*** an advisory for your code. It does not mean nobody has found anything.

BEYOND REACTIVE PATCHING

You can't out-scan or out-patch this on a monthly cycle

An SLA assumes a fix exists, and increasingly it doesn't. The answer is defense in depth, built for machine speed.

TRACK 1 · Ingestion defense

Harden the one boundary you fully own, the front door where third-party code enters. Cool-down windows, AI-driven scanning, provenance.

TRACK 2 · Ecosystem stewardship

Keep upstream alive, because no defensive layer produces a patch for an abandoned package. Disclose, fund, and adopt what matters.

REACHABILITY · CONTAINMENT *the foundation underneath both tracks*

TRACK 1

Ingestion Defense

Controls at the front door, the one boundary you fully own.

Most enterprises already proxy public registries through an internal mirror. **That chokepoint, one place every third-party artifact passes through, is where the controls go.**

INGESTION CONTROL #1

Cool-down windows

- **Don't admit a version the day it publishes.** Malicious releases are usually caught from outside the victim. Many within days.
- Pull immediately and you join the discovery process as a victim. Hold, and the rest of the ecosystem encounters it first.
- **Hold for several days to start.** Tune against your own release cadence.
- **Route churn vs. security releases differently.** A uniform hold widens the fix-deployed gap, so security fixes take the fast path. The exemption skips only the hold, so behavioral and provenance checks still run.

The exception path

Gate on an advisory in a public feed, matching the exact fixed version rather than the declared safe range. Plus a human-attested trigger (maintainer heads-up, embargo, your own finding).

⚠ A caching proxy is NOT a gate

Artifactory and Nexus fetch upstream metadata on demand, so a build still sees a brand-new version. Getting a real hold needs version-list filtering, coordinated with lockfiles.

INGESTION CONTROL #2

AI-driven malware & account-takeover scanning

The threat here is a package deliberately modified to attack whoever installs it, usually after account takeover.

Publishing anomalies

New signing key or a publisher that never released this package.

Install-time code

Obfuscated preinstall / postinstall / setup.py, running on laptops and build agents.

Behavior mismatch

A date library opening a network socket. The hardest to catch, since payloads sleep and evade.

Name confusion

Typosquats & dependency confusion. Fix the resolver, not just the public namespace.

Provenance (Sigstore, SLSA) answers origin, but it cannot answer safety. Under account takeover the real pipeline builds it, so the signature still verifies. Run all of this at ingestion as a gate, rather than as a report read later.

THE TRAP THAT UNDOES IT ALL

The auto-update trap

09:00 Compromised version publishes

09:40 Update bot opens a PR

10:15 Tests pass, functionality intact

PM Merged. Malicious code in your build before any advisory exists.

Fix: bot-side release-age delay (minimumReleaseAge / cooldown) + mirror filtering + human review on install-script changes.

UNDERNEATH BOTH TRACKS

Reachability: is the bug even reachable?

- **Scanners drown you.** The same SQL query is critical or harmless depending entirely on what reaches it. Pattern rules flag both.
- **Invert the direction.** Start where untrusted data enters, then trace where it can go. Entry points are what an attacker can touch.
- **Make it argue against itself.** A refutation stage tries to prove each finding wrong, favoring precision over recall and leaving a shorter list engineers actually read.
- **Honest limit:** framework wiring, queues, Java deserialization. Mark unknown, not unreachable.

Real refutations vs. impostors

Real: framework encoding neutralizes the payload, or an upstream type constraint makes the input impossible.

Impostors: authentication still leaves every insider, and compiler mitigations raise cost without removing the flaw.

Suppress with a record

Reasoning, an expiry, a re-check trigger, and a named owner who is not the pipeline.

UNDERNEATH BOTH TRACKS

Findings that arrive with proof

The red/green loop from TDD, aimed at a vulnerability

- a demonstration that triggers the bug
- a failing test encoding the missing property
- a fix that makes it pass

All in one reviewable pull request. A failing test is checkable in a way a CVSS score is not.

- **Evidence, not a specification.** A test covers the one input it triggers, so review the fix against the whole class. Partial fixes are how follow-up advisories happen.
- **Independent verification differs or it's theater.** A verifier from the same model family fails on the same inputs.
- **An SBOM tells you what is present, not what is exposed.** Feed it into the reachability query instead of patching in severity order.
- **VulnHunter** is Capital One's open-source take. The pattern is architectural, not product-specific.

TRACK 2

Ecosystem Stewardship

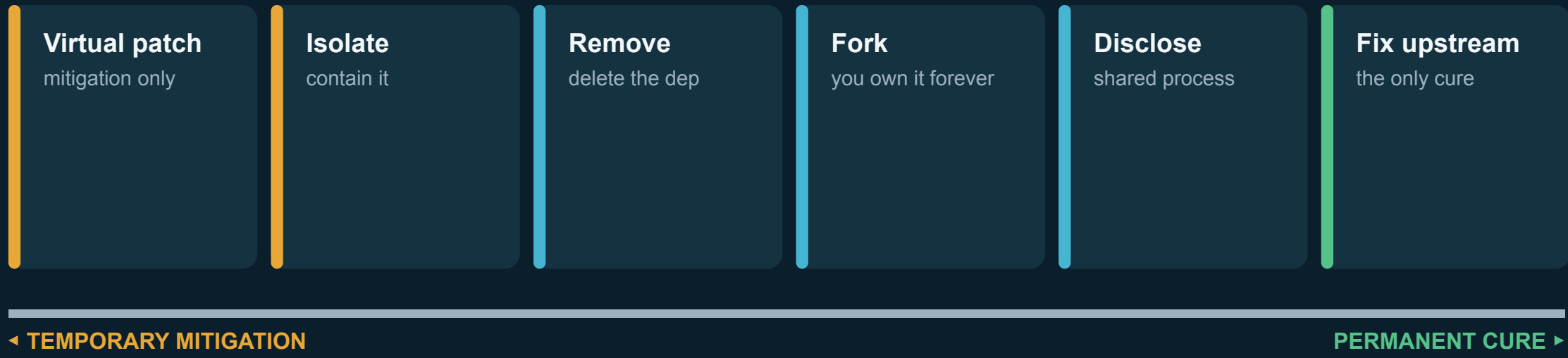
Fixing the code you do not own.

You can detect a forever-day and prove attacker input reaches it. **Then the work stops, because the fix belongs in a repo you don't control. Either you write it, or you make sure someone is still there to.**

WHEN UPSTREAM IS DEAD

Make waiting a choice

Six options, ordered from temporary mitigation to permanent cure.



Kill the anti-pattern first: “awaiting upstream fix”, reviewed monthly, escalated quarterly, waiting for a patch that has no author. A published advisory now feeds exploit generation, so coordinate disclosure timing.

KEEP THE MAINTAINER THERE

Disclose through a shared process · fund · adopt

Akrites (Linux Foundation, launched mid-2026, initial funding via Alpha-Omega)

Commits to deduplicating reports and routing one validated patch upstream on the maintainer's terms. Chartered as maintainer of last resort, and never publishes into someone else's namespace. Disclosure-process docs were still in draft at launch. A resource worth evaluating at akrites.org.

Fund by exposure, not downloads

The target list is a byproduct of the reachability analysis you already run. Size the ask against scanning-tool spend.

Limit: a sponsored maintainer owes no response time, so it is not a compensating control. Foundation-directed funding clears procurement.

Adopt the ones that matter

Standing engineering time, one day every two weeks, on triage, CI, and upstream security fixes.

Send fixes upstream by default. Security fixes are the exception. A public PR is itself a disclosure, so route it privately, and set a deadline for dormant projects.

UNDERNEATH BOTH TRACKS

Containment: assume the dependency is compromised

Zero trust, one level down, into the process rather than the network hop. A date library doesn't need to read every environment variable.

Short-lived creds

Narrowest change. Cuts what a stolen token is worth.

Egress policy

Deny by default. The hard part is destination inventory.

Scoped secrets

Mount only what a workload needs.

Sandbox risky code

WebAssembly or a hardened runtime. Contains memory corruption rather than preventing it.

Build agents first

The richer target, holding cloud creds, signing keys, and prod access. Ephemeral agents, OIDC with a scoped subject claim, hermetic builds.

The limit on credential controls

In-process code already holds the workload identity. Restrict what it's authorized to read, not only what's mounted.

Coding assistants that suggest dependencies

A suggested import arrives in a pull request reviewed for logic rather than supply-chain risk, so it skips every ingestion control.

The common version

Assistants recommend deprecated, unmaintained, or superseded packages, because training data reflects a past ecosystem rather than today's.

The stranger version

Assistants suggest packages that never existed. Attackers register the invented names, so the hallucination resolves to their code.

The controls are boring. A curated registry with an allowlist blocks an unvetted or invented name from installing. Each new dependency becomes its own review item, and the same ingestion checks run no matter the entry path.

PUTTING IT TOGETHER

Where to start

Sequenced so early items make later ones cheaper. **You can start items 1 and 2 this week.**

1	**Ingestion chokepoint**	Highest-value move, but a program. Run egress block log-only first. Add version filtering once routing exists.
2	**Bot delay, week one**	Set in the bot's own config. The smallest change on the list, and it closes the likeliest path in. Plus human review.
3	**Attack the triage problem**	Reachability analysis changes what engineers spend time on, and makes security credible with engineering.
4	**Contain what runs**	Build agents first (narrower, richer creds), then credential lifetime and secret scoping in the app tier.
5	**Start stewardship early**	Pays back last, but it's the only item that yields a patch for an abandoned package. Set up shared disclosure now.

WHAT HOLDS UP AT MACHINE SPEED

Each layer's gap is another layer's job

LAYER	COVERS	LEAVES AS A GAP
Ingestion	filters what enters	misses what's already installed
Reachability	ranks findings by real exposure	can't patch an abandoned package
Containment	limits what a compromise can do	doesn't remove the flaw
Stewardship	produces an actual patch	takes quarters to show results

Additive to the SLA. Together these layers cover the exposure a “fix it fast” deadline was never designed to reach.

A condition to design for rather than a wave to wait out.

THE CLOSING IDEA

Attackers get frontier capability off the shelf.

Defenders shouldn't have to build theirs from scratch.

The defensive tooling here, reachability analysis included, is increasingly open source by intent.

Thank you.



Stephen Husak · Staff Engineer, Capital One

Slides github.com/shusak/ato2026

LinkedIn linkedin.com/in/stevehusak

X x.com/shusak

Scan for the slides